

검색기반 AI (Search-based AI & RAG)

언어 모델이 세상을 검색하는 방법

본 수업에서는 검색기반 AI의 핵심 개념인 RAG(Retrieval-Augmented Generation)를 중심으로, 전통적 AI와의 차이점, 작동 원리, 실제 활용 사례, 그리고 미래 전망을 체계적으로 살펴봅니다.

기존 LLM의 한계 — "알고 있는 것"만 말할 수 있다

대형 언어 모델(LLM)은 학습 데이터의 시간적 한계와 환각(Hallucination) 문제를 본질적으로 가지고 있다.

3~27%

연구에 따르면 LLM의 환각 발생률은 질문 유형에 따라 최대 27%에 달함 (Ji et al., 2023)

■ 지식 단절 (Knowledge Cutoff)

GPT-4, Gemini 등 LLM은 특정 시점까지의 데이터만 학습하여 최신 정보 반영 불가

■ 환각 (Hallucination)

모델이 사실이 아닌 정보를 자신 있게 생성하는 현상 — 의료·법률 분야에서 치명적

■ 출처 불명확

어떤 근거로 답변했는지 추적 불가하여 신뢰성 저하

■ 도메인 특화 한계

기업 내부 문서, 비공개 데이터에 접근 불가하여 맞춤형 답변 제공 어려움

검색기반 AI란 무엇인가 — 기억이 아닌 검색으로 답한다

RAG(Retrieval-Augmented Generation)는 외부 지식 베이스를 실시간으로 검색하여 LLM의 응답 품질을 획기적으로 향상시키는 아키텍처다.

■ RAG는 정보 검색(Retrieval)과 텍스트 생성(Generation)을 결합하여, 모델이 답변 생성 전에 관련 문서를 먼저 검색하고 이를 컨텍스트로 활용하는 방식이다.

— Lewis et al., 2020 (Facebook AI Research)

핵심 구성 3요소

1. 검색기 (Retriever)

역할: 관련 문서 탐색

예시 기술:
BM25, Dense Retrieval

2. 지식 베이스 (Knowledge Base)

역할: 외부 문서 저장소

예시 기술:
벡터 DB, 문서 DB

3. 생성기 (Generator)

역할: 검색 결과 기반 답변 생성

예시 기술:
GPT-4, Claude, Gemini

RAG의 작동 원리 — 4단계 파이프라인

RAG는 질의 → 검색 → 증강 → 생성의 4단계 파이프라인으로 작동하며, 각 단계의 품질이 최종 답변의 정확도를 결정한다.

01

질의 처리 (Query Processing)

- 사용자 질문을 벡터 임베딩으로 변환
- 의미론적 유사도 계산 준비



02

문서 검색 (Document Retrieval)

- 벡터 데이터베이스에서 유사 문서 Top-K 추출
- 키워드 검색(BM25) + 의미 검색(Dense) 하이브리드 방식



03

컨텍스트 증강 (Context Augmentation)

- 검색된 문서를 프롬프트에 삽입
- 관련성 재순위화(Re-ranking) 적용



04

답변 생성 (Answer Generation)

- LLM이 증강된 컨텍스트를 바탕으로 최종 답변 생성
- 출처 인용 가능

벡터 임베딩과 유사도 검색 — 의미를 숫자로 표현하다

벡터 임베딩은 텍스트의 의미를 고차원 수치 공간에 표현하여, 의미적으로 유사한 문서를 수학적으로 탐색할 수 있게 한다.

■ 임베딩 (Embedding)

텍스트를 수백~수천 차원의 실수 벡터로 변환
(예: "강아지"와 "개"는 벡터 공간에서 가깝게 위치)

■ 코사인 유사도

두 벡터 간 각도로 의미적 유사성 측정
(값 범위: -1 ~ 1)

■ 벡터 데이터베이스

수백만 개의 벡터를 고속 탐색하는 특수 DB

주요 벡터 DB 비교

데이터베이스	특징	적합 용도
Pinecone	완전 관리형, 고성능	프로덕션 환경
Weaviate	오픈소스, 멀티모달	유연한 커스터마이징
Chroma	경량, 로컬 실행	개발/프로토타입
FAISS (Meta)	초고속, 오프라인	대규모 배치 처리

RAG vs. Fine-tuning — 언제 무엇을 선택할까

RAG와 파인튜닝은 상호 배타적이지 않으며, 지식의 동적 업데이트 필요성과 비용 구조에 따라 전략적으로 선택 또는 결합해야 한다.

기준	RAG (검색기반 생성)	Fine-tuning (미세조정)
지식 업데이트	실시간 가능 (문서만 추가/수정)	재학습 필요 (시간 및 리소스 소모)
비용	검색 인프라 및 벡터 DB 유지 비용	높은 모델 학습 및 컴퓨팅 비용
출처 추적	가능 (검색된 문서 기반으로 답변)	어려움 (모델 가중치에 내재 됨)
도메인 적응	문서 추가로 즉시 적용 가능	대규모 데이터 수집 후 학습 필요
환각 억제	효과적 (컨텍스트에 기반하여 답변 제한)	부분적 (여전히 환각 발생 가능성 존재)
응답 스타일 조정	제한적 (프롬프트 엔지니어링 의존)	매우 효과적 (특정 말투, 형식 학습 가능)

결론 및 전략적 선택

최신성·정확성: RAG · 말투/형식: Fine-tuning · 둘 다 필요: RAG + Fine-tuning

실제 활용 사례 — 산업 전반에 걸친 검색기반 AI

검색기반 AI는 의료, 법률, 금융, 교육 등 지식 집약적 산업에서 이미 실질적인 가치를 창출하고 있다.

의료 분야

- 최신 임상 가이드라인 기반 진단 보조 시스템
- 전자의무기록(EMR) 검색 및 요약

법률 분야

- 판례 검색 및 법령 해석 자동화 (예: Harvey AI)
- 계약서 검토 및 리스크 분석

금융 분야

- 실시간 재무 보고서 기반 투자 분석
- 규제 컴플라이언스 문서 검색

교육 분야

- 교재 기반 맞춤형 Q&A 튜터 시스템
- 학술 논문 검색 및 요약 (예: Perplexity AI, Elicit)

기업 내부

- 사내 지식 베이스 기반 직원 지원 챗봇
- 기술 문서 및 매뉴얼 검색 자동화

검색기반 AI의 주요 도전 과제

RAG 시스템의 성능은 검색 품질, 컨텍스트 길이 한계, 다국어 처리 등 여러 기술적 도전에 의해 제약받는다.

1. 검색 품질 문제

- 부정확한 청킹(Chunking)으로 인한 컨텍스트 단절
- 질의-문서 간 어휘 불일치 (Vocabulary Mismatch)

2. 컨텍스트 창 한계

- LLM의 최대 토큰 수 제한으로 대량 문서 처리 어려움
- "Lost in the Middle" 현상: 중간 위치 정보 망각

3. 다국어 및 멀티모달 처리

- 한국어 등 비영어권 언어의 임베딩 품질 저하
- 이미지·표·PDF 등 비정형 데이터 처리 복잡성

4. 지연 시간(Latency)

- 검색 + 생성 이중 처리로 응답 속도 저하
- 실시간 서비스에서의 성능 최적화 필요

Advanced RAG — 성능을 높이는 최신 기법들

Beyond Naive RAG

Naive RAG의 한계를 극복하기 위해 쿼리 재작성, 하이브리드 검색, 재순위화 등 고급 기법들이 등장하여 RAG의 정확도와 효율성을 크게 향상시키고 있다.

■ 쿼리 재작성 (Query Rewriting)

사용자 질의를 더 검색 친화적인 형태로 변환하여 검색 정확도 향상

■ 재순위화 (Re-ranking)

Cross-encoder 모델로 검색 결과 재정렬하여 정밀도(Precision) 향상

■ GraphRAG (Microsoft)

지식 그래프 기반 검색으로 복잡한 다단계 추론 지원

■ 하이브리드 검색 (Hybrid Search)

BM25(키워드) + Dense(의미) 검색 결합으로 재현율(Recall) 향상

■ Self-RAG

모델이 스스로 검색 필요성을 판단하고 결과를 비판적으로 평가

■ Corrective RAG (CRAG)

검색 결과의 신뢰도를 평가하고 필요시 웹 검색으로 보완

검색기반 AI 구현 실습 — 핵심 도구와 프레임워크

LangChain, LlamaIndex 등 오픈소스 프레임워크를 활용하면 RAG 시스템을 빠르게 구축할 수 있으며, 실습을 통해 핵심 개념을 체화할 수 있다.

주요 프레임워크

프레임워크	특징	학습 난이도
LangChain	가장 널리 사용, 풍부한 생태계	중간
LlamaIndex	문서 인덱싱 특화, 직관적 API	낮음
Haystack	엔터프라이즈 중심, 파이프라인 구성	중간
DSPy	프로그래밍 방식의 프롬프트 최적화	높음

간단한 RAG 구현 흐름 (LangChain 기준)

```
# 1. 문서 로드 및 청킹
loader = PyPDFLoader("document.pdf")
chunks = text_splitter.split_documents(loader.load())

# 2. 벡터 DB 구축
vectorstore = Chroma.from_documents(chunks, embedding_model)

# 3. RAG 체인 구성
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
rag_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever
)
```

검색기반 AI의 윤리와 책임 — 신뢰할 수 있는 AI를 위하여

검색기반 AI는 출처 추적 가능성을 높이지만, 편향된 지식 베이스, 저작권 문제, 프라이버시 침해 등 새로운 윤리적 도전을 수반한다.

- 출처 편향

지식 베이스의 편향이 AI 답변에 그대로 반영될 수 있음

- 저작권 문제

검색된 문서의 저작권 보호 범위와 인용 기준 불명확

- 프라이버시

개인정보가 포함된 문서의 벡터화 및 저장 시 데이터 보호 필요

- 과신 위험

출처가 있다고 해서 반드시 정확한 것은 아님 — 비판적 검토 필수

- 접근성 불평등

고품질 RAG 시스템 구축 비용으로 인한 정보 격차 심화 가능성

핵심 원칙

1 투명성 (Transparency)

2 검증 가능성 (Verifiability)

3 책임성 (Accountability)

검색기반 AI의 미래 — 에이전트 AI와의 융합

검색기반 AI는 단순 Q&A를 넘어 자율적으로 계획하고 행동하는 AI 에이전트의 핵심 인프라로 진화하고 있다.

- **Agentic RAG**

AI 에이전트가 복잡한 목표를 위해 자율적으로 검색 전략을 수립하고 반복 실행

- **멀티모달 RAG**

텍스트를 넘어 이미지, 영상, 오디오까지 통합 검색

- **실시간 웹 검색 통합**

Perplexity AI, ChatGPT Search처럼 인터넷을 실시간 지식 베이스로 활용

- **개인화 RAG**

개인의 문서·이메일·캘린더를 기반으로 한 초개인화 AI 어시스턴트

- **연합 학습 + RAG**

프라이버시를 보호하면서 분산된 지식 베이스 활용

글로벌 RAG 시장 전망

\$11B (2030년 예상)

2023년 약 9억 달러에서 연평균 43.3% 성장 예상

검색기반 AI는 "더 똑똑한 AI"가 아니라
"더 정직하고 검증 가능한 AI"를 향한 여정이다.